

Lesson 14

Topic: DAX Optimization

Prerequisites: Download Chocolate Sales.csv file and disable Auto Date/Time options by Options in Current File and create new DATE table by using Calendarauto and Format functions.

###Basic/Intermediate/Advanced Level (1–10)

1) Use variable to calculate % Growth in Sales Compared to Last Year
Prerequisites Setup (must be done before DAX):

1. Load the dataset

Import Chocolate Sales.csv into Power BI using:

- Home > Get Data > Text/CSV

2. Disable Auto Date/Time (Optimization)

- File > Options and Settings > Options
- Under Current File > Data Load
- Uncheck Auto Date/Time for new files

3. Create a separate Date Table:

Go to Modeling > New Table, and enter:

dax

Date =

```
ADDCOLUMNS(
    CALENDARAUTO(),
    "Year", YEAR([Date]),
    "Month Number", MONTH([Date]),
    "Month Name", FORMAT([Date], "MMMM"),
    "YearMonth", FORMAT([Date], "YYYY-MM")
)
```

Then, in Model View, link Date[Date] to the sales table's OrderDate or similar date column.

Goal: Calculate % Growth in Sales vs Last Year using VAR

Step-by-step Measure

dax

Sales Growth % YoY =

VAR CurrentSales = CALCULATE(SUM('Chocolate Sales'[Sales]))

VAR LastYearSales =

CALCULATE(
SUM('Chocolate Sales'[Sales]),
SAMEPERIODLASTYEAR('Date'[Date])
)

VAR Growth =

DIVIDE(CurrentSales - LastYearSales, LastYearSales)

RETURN

Growth

Explanation:

- VAR CurrentSales: total sales for the current context (e.g., current month/year in visual).
- VAR LastYearSales: gets last year's value using SAMEPERIODLASTYEAR.
- DIVIDE(): safer alternative to / to avoid division by zero.
- Final Result: % change compared to the same period last year.

Add this Measure to:

- A Matrix visual with Date[Year] and Date[Month Name] or YearMonth
- Add both:
 - Sales
 - Sales Growth % YoY

2) Use variable to calculate the difference between Sales Amount of current month and previous month

Step-by-step Measure Using VAR:

dax

Sales Difference MoM =

VAR CurrentMonthSales =

```
CALCULATE(  
    SUM('Chocolate Sales'[Amount])  
)
```

VAR PrevMonthSales =

```
CALCULATE(  
    SUM('Chocolate Sales'[Amount]),  
    PREVIOUSMONTH('Date'[Date])  
)
```

RETURN

CurrentMonthSales - PrevMonthSales

Explanation:

Part Purpose

CurrentMonthSales Sum of sales for the current filter context (e.g., Feb 2023)

PrevMonthSales Sum of sales for the previous month using PREVIOUSMONTH()

RETURN Simple subtraction to get the difference

Example Matrix:

Use in Matrix/Table with:

Rows: Date[YearMonth] or Date[Month Name]

Values:

SUM(Amount)

Sales Difference MoM

Optional: Add a Change Direction Indicator

dax

Sales Difference Icon =

VAR Diff = [Sales Difference MoM]

RETURN

SWITCH(TRUE(),

Diff > 0, "↑",

Diff < 0, "↓",

"→"

)

Notes:

Make sure your Date table is continuous (no missing months).

Ensure the relationship is set: 'Date'[Date] → 'Chocolate Sales'[Date]

3) Calculate total boxes shipped and average monthly boxes in one measure using VAR

inal DAX Measure:

dax

Boxes Summary =

VAR TotalBoxes =

SUM('Chocolate Sales'[Boxes Shipped])

VAR DistinctMonths =

```

    CALCULATE(
        DISTINCTCOUNT('Date'[YearMonth])
    )
VAR AvgBoxesPerMonth =
    DIVIDE(TotalBoxes, DistinctMonths)
RETURN
    "Total: " & FORMAT(TotalBoxes, "#,##0") &
    " | Monthly Avg: " & FORMAT(AvgBoxesPerMonth, "#,##0.00")

```

How it works:

Part	Description
TotalBoxes	Total of all boxes shipped (filtered context applies)
DistinctMonths	Counts how many unique months are visible in context
DIVIDE()	Safe division
RETURN	A single text output combining both values in a readable format

Output Example in Table Visual:

Country	Boxes Summary
India	Total: 8,000 Monthly Avg: 1,333.33
UK	Total: 4,200 Monthly Avg: 700.00

Requirements:

Make sure your Date table includes a YearMonth column:

dax

```
YearMonth = FORMAT([Date], "YYYY-MM")
```

Ensure you have a relationship between 'Chocolate Sales'[Date] and Date[Date].

Optional: Separate Measures (Advanced)

If you prefer to show them as numbers instead of text, create two separate measures:

dax

```

Total Boxes = SUM('Chocolate Sales'[Boxes Shipped])
Average Monthly Boxes =
VAR TotalBoxes = SUM('Chocolate Sales'[Boxes Shipped])
VAR DistinctMonths =
CALCULATE(DISTINCTCOUNT('Date'[YearMonth]))
RETURN DIVIDE(TotalBoxes, DistinctMonths)

```

4) Calculate total boxes shipped and average monthly boxes in one measure using VAR and return average monthly boxes.

Final Optimized DAX Measure

dax

Average Monthly Boxes =

VAR TotalBoxes =

SUM('Chocolate Sales'[Boxes Shipped])

VAR DistinctMonths =

CALCULATE(

DISTINCTCOUNT('Date'[YearMonth])

)

VAR AvgBoxesPerMonth =

DIVIDE(TotalBoxes, DistinctMonths)

RETURN

AvgBoxesPerMonth

Explanation:

Variable	Purpose
----------	---------

TotalBoxes	Total boxes shipped in the current filter context
------------	---

DistinctMonths How many months exist in that context (e.g. Year or Country)

AvgBoxesPerMonth Main result = average per month

RETURN Returns only the average monthly boxes as a numeric result (can be used in charts/tables)

Use this measure in a visual like:

A Table/Matrix by Country, Sales Person, or Date[Year]

A Card visual to show company-wide monthly average

A Line chart to show trends over months

Important:

Make sure you have this column in your Date table:

dax

YearMonth = FORMAT([Date], "YYYY-MM")

And ensure:

'Date'[Date] is connected to 'Chocolate Sales'[Date]

Your Date table spans the full range of order dates

5) Calculate growth percentage from last month.

To calculate the **growth percentage from last month** using DAX, you'll use variables and PREVIOUSMONTH() to compare the current month's value with the previous month's.

Measure: Monthly Growth % (MoM)

dax

Monthly Sales Growth % =

VAR CurrentMonthSales =

CALCULATE(SUM('Chocolate Sales'[Amount]))

VAR PrevMonthSales =

CALCULATE(
SUM('Chocolate Sales'[Amount]),
PREVIOUSMONTH('Date'[Date])
)

RETURN

DIVIDE(CurrentMonthSales - PrevMonthSales, PrevMonthSales)

Explanation:

Step	Logic
CurrentMonthSales	Sales for the current filter context (e.g. April 2023)
PrevMonthSales	Sales for previous month using PREVIOUSMONTH()
DIVIDE()	Safely divides the difference by previous month's sales to get growth %

How to Use:

- Place this measure in a **Matrix/Table visual**
- Use Date[YearMonth] or Date[Month Name] in **rows**
- Add this measure to **values** next to Amount

Output Example:

YearMonth	Sales	Growth %
Jan 2023	\$10,000	—
Feb 2023	\$12,000	20.00%

YearMonth	Sales	Growth %
Mar 2023	\$9,000	-25.00%

Notes:

- Format the measure as **percentage** in Power BI.
- Ensure your Date table includes a column like:
dax
YearMonth = FORMAT([Date], "YYYY-MM")
- Confirm relationship: 'Date'[Date] → 'Chocolate Sales'[Date]

6) Create a moving average of sales over the last 3 months.

Measure: 3-Month Moving Average of Sales

dax

Sales Moving Avg (3 Months) =

VAR CurrentDate = MAX('Date'[Date])

VAR Last3Months =

 DATESINPERIOD(

 'Date'[Date],

 CurrentDate,

 -3,

 MONTH

)

RETURN

CALCULATE(

 AVERAGEX(

 Last3Months,

 CALCULATE(SUM('Chocolate Sales'[Amount])))

)

)

Explanation:

Part	Description
MAX('Date'[Date])	Gets the latest date in the current context (e.g., Feb 2023)
DATESINPERIOD()	Returns a 3-month window ending at the current date
AVERAGEX()	Iterates over the 3 months and calculates the average sales

How to Visualize:

1. Use a **Line Chart** or **Matrix/Table**.
 2. X-axis or Rows: Date[YearMonth]
 3. Values:
 - SUM(Amount)
 - Sales Moving Avg (3 Months) (this measure)
- 7) Use Card to show a Dynamic Message Based on Sales Rank and YoY Performance. For each chocolate product show a message like:
- "Top Performer - Sales up by X%"
 - "Consistent Performer"
 - "Needs Improvement"
- Ps:(use selectedvalue, rankx, and Time Intelligence functions)

Step-by-Step DAX Measure for the Card

dax

CopyEdit

Performance Message =

VAR SelectedProduct = SELECTEDVALUE('Chocolate Sales'[Product])

-- Calculate current year sales

VAR CY_Sales =

CALCULATE(

SUM('Chocolate Sales'[Amount]),

DATESYTD('Date'[Date]))

)

-- Calculate last year sales

VAR LY_Sales =

CALCULATE(

SUM('Chocolate Sales'[Amount]),

DATESYTD(SAMEPERIODLASTYEAR('Date'[Date])))

)

-- Calculate YoY % growth

VAR YoY_Growth = DIVIDE(CY_Sales - LY_Sales, LY_Sales)

-- Rank of current product by CY sales

VAR ProductRank =

RANKX(

ALL('Chocolate Sales'[Product]),

CALCULATE(SUM('Chocolate Sales'[Amount])),

,

DESC

)

-- Dynamic message logic

VAR Result =

SWITCH(

TRUE(),

ProductRank <= 3 && YoY_Growth > 0,

"Top Performer - Sales up by " & FORMAT(YoY_Growth, "0.0%"),

YoY_Growth >= 0,

"Consistent Performer",

YoY_Growth < 0,

"Needs Improvement"

)

RETURN

Result

What It Does:

Logic

Rank $\leq$ 3 and YoY Growth $>$ 0

Growth $\geq$ 0 but not top 3

Growth $<$ 0

Message

"Top Performer - Sales up by X%"

"Consistent Performer"

"Needs Improvement"

How to Use in a Card:

1. Insert a **Card** visual.
2. Add the Performance Message measure.
3. Add a **slicer or filter** for 'Chocolate Sales'[Product].

As you change the selected product, the card updates dynamically.

Example Output:

Product	Card Message
Mint Chip Choco	Top Performer - Sales up by 12.5%
85% Dark Bars	Consistent Performer
Peanut Butter Cubes	Needs Improvement

8) List Top 5 tips to optimize DAX query manually and explain why you choose.

Top 5 Manual DAX Optimization Tips (with reasons)

1. Use VAR to Store and Reuse Expressions

Why:

- Reduces **repeated calculation** of the same logic
- Makes code **cleaner and faster**

Example:

dax

VAR TotalSales = SUM(Sales[Amount])

RETURN TotalSales * 0.1

Instead of calculating SUM(Sales[Amount]) multiple times, store it once.

2. Avoid Row Context Iterators (FILTER, SUMX, RANKX) on Large Tables Without Reducing the Scope

Why:

- Iterators process **row-by-row**, which can be very slow on big tables.
- Always **filter the table** down before iterating.

Better Practice:

dax

-- BAD:

```
RANKX(ALL(Sales), SUM(Sales[Amount]))
```

-- GOOD:

```
RANKX(  
    FILTER(ALL(Sales), Sales[Region] = "Europe"),  
    CALCULATE(SUM(Sales[Amount]))  
)
```

Helps reduce evaluation scope = faster calculations.

3. Use **DIVIDE()** Instead of /

Why:

- Handles **division-by-zero** safely
- Avoids errors or blank visuals

Example:

dax

CopyEdit

```
DIVIDE(CurrentSales, PrevSales)
```

Prevents crashes or blank results when denominator is 0.

4. Minimize Use of **CALCULATE** with Complex Filters Inside Measures

Why:

- Every **CALCULATE()** modifies the filter context, which can be expensive.
- Keep logic **as simple and flat** as possible.

Example:

Instead of:

dax

```
CALCULATE(SUM(Sales[Amount]), Sales[Country] = "USA")
```

Use:

dax

FILTER(Sales, Sales[Country] = "USA") inside CALCULATE once

5. Create and Use a Dedicated, Marked Date Table

Why:

- Enables full use of **Time Intelligence functions** (SAMEPERIODLASTYEAR, DATESYTD, etc.)
- Avoids bugs from Auto Date/Time tables
- Greatly improves **performance and reliability**

Example:

dax

Date = CALENDARAUTO()

Then:

- Mark as **Date Table**
- Create relationships
- Use Date[Date] in visuals

9) What is the benefit of using DAX optimization tools like DAX Studio, Performance Analyzer, Tabular Editor

Benefits of Using DAX Optimization Tools

1. DAX Studio

A free, external tool for analyzing and tuning DAX performance.

Key Benefits:

- Shows **query plans and timings** to identify slow-performing measures.
- Tracks **storage engine vs. formula engine** time (SE vs. FE).
- Can run **server timings** and **benchmark measures**.
- Allows writing and testing **DAX queries** outside of Power BI.
- Can identify **unused columns and tables** using DMV queries.

Why it's valuable:

You can **pinpoint bottlenecks** in your DAX, like heavy iterators or inefficient filters.

2. Performance Analyzer (built into Power BI)

Helps monitor what happens when visuals load and measures run.

Key Benefits:

- See **how long each visual** takes to render.
- Breaks down time into:
 - **DAX Query time**
 - **Visual display time**
 - **Other** (data fetch, etc.)
- Copy DAX from slow visuals to test in DAX Studio.

Why it's valuable:

Quickly identify which **visuals or measures are causing slowness** — especially useful in dashboards with many cards or slicers.

3. Tabular Editor (2.x free, 3.x paid)

A powerful modeling tool for editing Power BI and SSAS models.

Key Benefits:

- Quickly edit and create **measures, calculated columns, and format strings**.
- Create and apply **calculation groups** (e.g., for dynamic time intelligence).
- Bulk edit DAX expressions.
- Validate model against **best practices** using built-in or custom rules.
- Organize measures into **folders** for better structure.

Why it's valuable:

Enables **advanced modeling**, automation, and better maintainability in large models.

Summary Table

Tool	Primary Use	Key Benefits
DAX Studio	Query & performance analysis	Deep insights into query time, bottlenecks, storage vs. formula engine
Performance Analyzer	Visual load diagnostics	Quickly shows slow visuals/measures in Power BI
Tabular Editor	Modeling and automation	Advanced modeling, measure editing, best-practice checks

When to Use Each?

Use Case	Tool
Optimize slow visuals	Performance Analyzer + DAX Studio
Refactor long DAX measures	Tabular Editor
Find performance bottlenecks	DAX Studio (Server Timings)
Create dynamic time logic (YoY, MoM)	Tabular Editor Calculation Groups

- 10) Create a flag (Yes/No) if a product is in the top 5 by total sales. Use RANKX in a variable; avoid calculating rank more than once.**

Create a measure or column that shows:

- "Yes" → if product is in Top 5 by total sales
- "No" → if not

Measure: Product in Top 5 (Yes/No)

dax

Top 5 Product Flag =

VAR ProductSales =

CALCULATE(SUM('Chocolate Sales'[Amount]))

VAR ProductRank =

RANKX(

ALL('Chocolate Sales'[Product]),

CALCULATE(SUM('Chocolate Sales'[Amount])),

,

DESC

)

RETURN

IF(ProductRank <= 5, "Yes", "No")

Explanation:

Step	Description
------	-------------

ProductSales	Gets total sales for the current product
--------------	--

ProductRank	Calculates rank once using RANKX over all products
-------------	---

IF()	Returns "Yes" if rank ≤ 5 , otherwise "No"
------	---

This avoids re-calculating the rank multiple times — it's done once and reused.

How to Use:

1. Place this measure in a **Matrix** or **Table** with:
 - Rows: Product
 - Values: Total Sales, Top 5 Product Flag
2. Or use as a **filter** (e.g., filter visuals to only show "Yes" products).